

Week 2

1. Initialised project repository with a modular directory structure separating service logic, utilities, testing tools, and documentation.
2. Configured Python 3.11 environment and dependency management via `requirements.txt`, ensuring compatibility with BullMQ asyncio.
3. Implemented environment-driven configuration using `.env` with:
 - Gemini API key
 - Redis configuration
 - AWS credentials
 - Cache settings
4. Integrated structured logging using `colorlog`, implementing timestamped, colour-coded logs for better traceability.
5. Created Dockerfile and docker-compose setup to orchestrate:
 - Redis
 - Worker service
 - Redis Commander
6. Verified Redis connectivity and tested message queue creation manually.
7. Built base service skeleton (`avatar_image_service.py`) with service startup, async loop handling, and graceful shutdown scaffolding.